

Core Language Working Group "ready" Issues for the June, 2018 (Rapperswil) meeting

Section references in this document reflect the section numbering of document [WG21 N4700](#).

2254. Standard-layout classes and bit-fields

Section: 12.2 [class.mem] **Status:** ready **Submitter:** Richard Smith **Date:** 2016-03-23

According to 12.2 [class.mem] paragraph 25,

If a standard-layout class object has any non-static data members, its address is the same as the address of its first non-static data member. Otherwise, its address is the same as the address of its first base class subobject (if any). [*Note:* There might therefore be unnamed padding within a standard-layout struct object, but not at its beginning, as necessary to achieve appropriate alignment. —*end note*] [*Note:* The object and its first subobject are pointer-interconvertible (6.9.2 [basic.compound], 8.2.9 [expr.static.cast]). —*end note*]

This wording does not consider the case when the first non-static data member is a bit-field, which cannot have its address taken.

Proposed resolution, February, 2018: [SUPERSEDED]

Change 12.2 [class.mem] paragraph 25 as follows:

If a standard-layout class object has any non-static data members, its address is the same as the address of its first non-static data member **if that member is not a bit-field**. ~~Otherwise, its~~ **Its** address is **also** the same as the address of **each of** its ~~first~~ base class subobjects ~~(if any)~~. [*Note:* There might therefore be unnamed padding within a standard-layout struct object **inserted by an implementation**, but not at its beginning, as necessary to achieve appropriate alignment. —*end note*] [*Note:* The object and its first subobject are pointer-interconvertible (6.9.2 [basic.compound], 8.2.9 [expr.static.cast]). —*end note*]

Notes from the March, 2018 meeting:

It was pointed out that the definition of pointer interconvertibility in 6.7.2 [basic.stc.thread] paragraph 4 refers to “the first base class subobject” of the object and must also be updated to reflect the above proposed resolution.

Proposed resolution (March, 2018):

1. Change 6.9.2 [basic.compound] bullet 4.3 as follows:

Two objects *a* and *b* are *pointer-interconvertible* if:

- ...
- one is a standard-layout class object and the other is the first non-static data member of that object, or, if the object has no non-static data members, ~~the first~~ **any** base class subobject of that object (12.2 [class.mem]), or
- ...

2. Change 12.2 [class.mem] paragraph 25 as follows:

If a standard-layout class object has any non-static data members, its address is the same as the address of its first non-static data member **if that member is not a bit-field**. ~~Otherwise, its~~ **Its** address is **also** the same as the address of **each of** its ~~first~~ base class subobjects ~~(if any)~~. [Note: There might therefore be unnamed padding within a standard-layout struct object **inserted by an implementation**, but not at its beginning, as necessary to achieve appropriate alignment. —end note] [Note: The object and its first subobject are pointer-interconvertible (6.9.2 [basic.compound], 8.2.9 [expr.static.cast]). —end note]

2293. Requirements for *simple-template-id* used as a *class-name*

Section: 12 [class] **Status:** ready **Submitter:** CWG **Date:** 2016-06-22

There is currently no requirement that a *simple-template-id* used as a *class-name* (12 [class] paragraph 1) must have template arguments for every template parameter that does not have a default template argument.

Proposed resolution (March, 2018):

1. Change 6.3.1 [basic.scope.declarative] paragraph 1 as follows:

Every name is introduced in some portion of program text called a declarative region, which is the largest part of the program in which that name is ~~valid~~ **valid**, that is, in which that name may be used as an unqualified name to refer to the same entity. In general...

2. Add the following as a new paragraph after 17.2 [temp.names] paragraph 7:

A template-id that names an alias template specialization is a type-name.

A template-id is valid if

- **there are at most as many arguments as there are parameters or a parameter is a template parameter pack (17.6.3 [temp.variadic]),**
- **there is an argument for each non-deducible non-pack parameter that does not have a default template-argument,**
- **each template-argument matches the corresponding template-parameter (17.3 [temp.arg]),**
- **substitution of each template argument into the following template parameters (if any) succeeds, and**

- if the *template-id* is non-dependent, the associated constraints are satisfied as specified in the next paragraph.

A *simple-template-id* shall be valid unless it names a function template specialization (17.9.2 [temp.deduct]). [Example:

```
template<class T, T::type n = 0> class X;
struct S {
    using type = int;
};
using T1 = X<S, int, int>; // error: too many arguments
using T2 = X<>;           // error: no default argument for first template parameter
using T3 = X<1>;          // error: value 1 does not match type-parameter
using T4 = X<int>;         // error: substitution failure for second template parameter
using T5 = X<S>;          // OK
```

—end example]

3. Change 17.9.2 [temp.deduct] paragraph 2 as follows, converting from bullets to running text:

When an explicit template argument list is specified, if the template arguments are not compatible with the template parameter list or do not result in a valid function type as described below **given *template-id* is invalid (17.2 [temp.names])**, type deduction fails. Specifically, the following steps are performed when evaluating an explicitly specified template argument list with respect to a given function template:

- If the specified template arguments do not match the template parameters in kind (i.e., type, non-type, template), or if there are more arguments than there are parameters and no parameter is a template parameter pack, or if there is not an argument for each non-pack parameter, type deduction fails.
- If any non-type argument does not match the type of the corresponding non-type template parameter, and is not convertible to the type of the corresponding non-type parameter as specified in 17.3.2 [temp.arg.nontype], type deduction fails.
- The specified template argument values are substituted for the corresponding template parameters as specified below.

2294. Dependent auto static data members

Section: 17.7.2.2 [temp.dep.expr] **Status:** ready **Submitter:** Richard Smith **Date:** 2016-06-22

The list of type-dependent *id-expressions* in 17.7.2.2 [temp.dep.expr] paragraph 3 should include the case when a static data member is declared with the auto type specifier and the initializer is dependent.

Proposed resolution, March, 2018:

Add the following as a new bullet after 17.7.2.2 [temp.dep.expr] bullet 3.2:

- ...

- an *identifier* associated by name lookup with a non-type *template-parameter* declared with a type that contains a placeholder type (10.1.7.4 [dcl.spec.auto]),
- **an *identifier* associated by name lookup with a variable declared with a type that contains a placeholder type (10.1.7.4 [dcl.spec.auto]) where the initializer is type-dependent,**
- ...

2321. Conditional operator and cv-qualified class prvalues

Section: 8.16 [expr.cond] **Status:** ready **Submitter:** Richard Smith **Date:** 2016-09-06

The following example is ill-formed:

```
struct A {};
struct B : A {};
using T = const B;
A a = true ? A() : T();
```

We don't convert from A to T because we can't form an implicit conversion sequence. We don't convert from T to A because T is more cv-qualified (even though we could form an implicit conversion sequence). It would seem reasonable to accept this case; it seems that we should only be using cv-qualifiers as a tie-breaker if the class types are otherwise the same.

Proposed resolution (March, 2018):

Change 8.16 [expr.cond] bullet 4.3 as follows:

...Attempts are made to form an implicit conversion sequence from an operand expression E1 of type T1 to a target type related to the type T2 of the operand expression E2 as follows:

- ...
- If E2 is a prvalue or if neither of the conversion sequences above can be formed and at least one of the operands has (possibly cv-qualified) class type:
 - if T1 and T2 are the same class type (ignoring cv-qualification), ~~or one is a base class of the other,~~ and T2 is at least as cv-qualified as T1, the target type is T2,
 - **otherwise, if T2 is a base class of T1, the target type is cv1 T2, where cv1 denotes the cv-qualifiers of T1,**
 - otherwise, the target type is the type that E2 would have after applying the lvalue-to-rvalue (7.1 [conv.lval]), array-to-pointer (7.2 [conv.array]), and function-to-pointer (7.3 [conv.func]) standard conversions.

2322. Substitution failure and lexical order

Section: 17.9.2 [temp.deduct] **Status:** ready **Submitter:** Xiang Fan **Date:** 2016-09-06

According to 17.9.2 [temp.deduct] paragraph 7,

The substitution occurs in all types and expressions that are used in the function type and in template parameter declarations. The expressions include not only constant expressions such as those that appear in array bounds or as nontype template arguments but also general expressions (i.e., non-constant expressions) inside `sizeof`, `decltype`, and other contexts that allow non-constant expressions. The substitution proceeds in lexical order and stops when a condition that causes deduction to fail is encountered.

However, the same type can be represented in different lexical orders. For example, there is implementation variance on the following example, presumably because of preferring different declarations:

```
template <class T> struct A { using X = typename T::X; };

template <class T> typename T::X f(typename A<T>::X);
template <class T> auto f(typename A<T>::X) -> typename T::X;
template <class T> void f(...) { }

void h() {
    f<int>(0);
}
```

Proposed resolution, March, 2018:

Change 17.9.2 [temp.deduct] paragraph 7 as follows:

The substitution occurs in all types and expressions that are used in the function type and in template parameter declarations. The expressions include not only constant expressions such as those that appear in array bounds or as nontype template arguments but also general expressions (i.e., non-constant expressions) inside `sizeof`, `decltype`, and other contexts that allow non-constant expressions. The substitution proceeds in lexical order and stops when a condition that causes deduction to fail is encountered. **If substitution into different declarations of the same function template would cause template instantiations to occur in a different order or not at all, the program is ill-formed; no diagnostic required.** [Note: The equivalent substitution in exception specifications is done only when the *noexcept-specifier* is instantiated, at which point a program is ill-formed if the substitution results in an invalid type or expression. —end note] [Example:

```
template <class T> struct A { using X = typename T::X; };
template <class T> typename T::X f(typename A<T>::X);
template <class T> void f(...) { }
template <class T> auto g(typename A<T>::X) -> typename T::X;
template <class T> void g(...) { }
template <class T> typename T::X h(typename A<T>::X);
template <class T> auto h(typename A<T>::X) -> typename T::X; // redeclaration
template <class T> void h(...) { }

void h x() {
    f<int>(0);    // OK, substituting return type causes deduction to fail
    g<int>(0);    // error, substituting parameter type instantiates A<int>
    h<int>(0);    // ill-formed, no diagnostic required
}
```

—end example]

2339. Underspecified template arguments in structured bindings

Section: 11.5 [dcl.struct.bind] **Status:** ready **Submitter:** Daveed Vandevoorde **Date:** 2017-03-29

According to 11.5 [dcl.struct.bind] paragraph 3,

Otherwise, if the *qualified-id* `std::tuple_size<E>` names a complete type, the expression `std::tuple_size<E>::value` shall be a well-formed integral constant expression and the number of elements in the *identifier-list* shall be equal to the value of that expression. The *unqualified-id* `get` is looked up in the scope of `E` by class member access lookup (6.4.5 [basic.lookup.classref]), and if that finds at least one declaration, the initializer is `e.get<i>()`. Otherwise, the initializer is `get<i>(e)`, where `get` is looked up in the associated namespaces (6.4.2 [basic.lookup.argdep]). In either case, `get<i>` is interpreted as a *template-id*.

It is not clear what `i` is in this description, and in particular, its type is not specified.

Proposed resolution, March, 2018:

Change 11.5 [dcl.struct.bind] paragraph 3 as follows:

Otherwise, if the *qualified-id* `std::tuple_size<E>` names a complete type, the expression `std::tuple_size<E>::value` shall be a well-formed integral constant expression and the number of elements in the *identifier-list* shall be equal to the value of that expression. **Let `i` be an index prvalue of type `std::size_t` corresponding to `vi`.** The *unqualified-id* `get` is looked up in the scope of `E` by class member access lookup (6.4.5 [basic.lookup.classref]), and if that finds at least one declaration, the initializer is `e.get<i>()`. Otherwise, the initializer is `get<i>(e)`, where `get` is looked up in the associated namespaces (6.4.2 [basic.lookup.argdep]). In either case, `get<i>` is interpreted as a *template-id*. [Note: Ordinary unqualified lookup...